INSTITUCIÓN UNIVERSITARIA
PASCUAL BRAVO
Acreditados en Alta Calidad

Formar
Servir
Cuidar

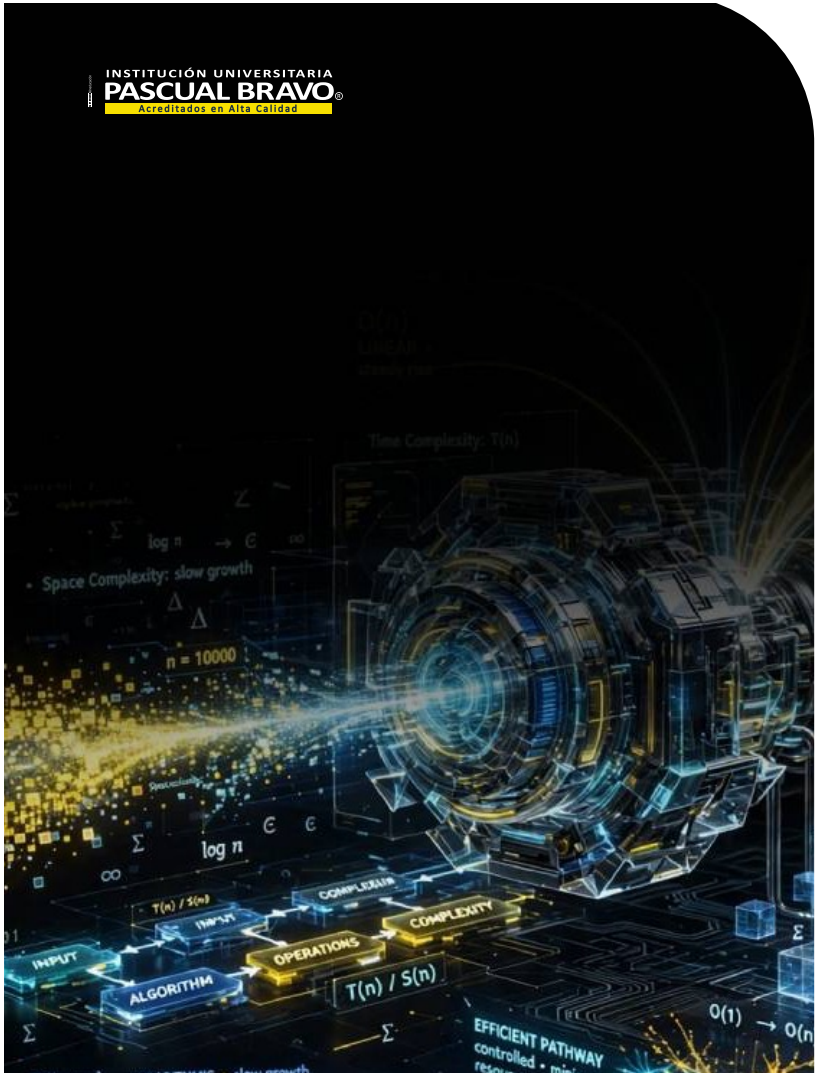
Complejidad computacional



"El hecho de que un problema sea decidible no significa que los algoritmos que tenemos para resolverlo sean eficientes en la práctica."

Complexity Theory
https://web.stanford.edu/class/archive/cs/cs103/cs103.1256/lectures/26/

https://web.stanford.edu/class/archive/cs/cs103/cs103.1256/lectures/26/
https://docs.python.org/3.16/library/time-complexity.html

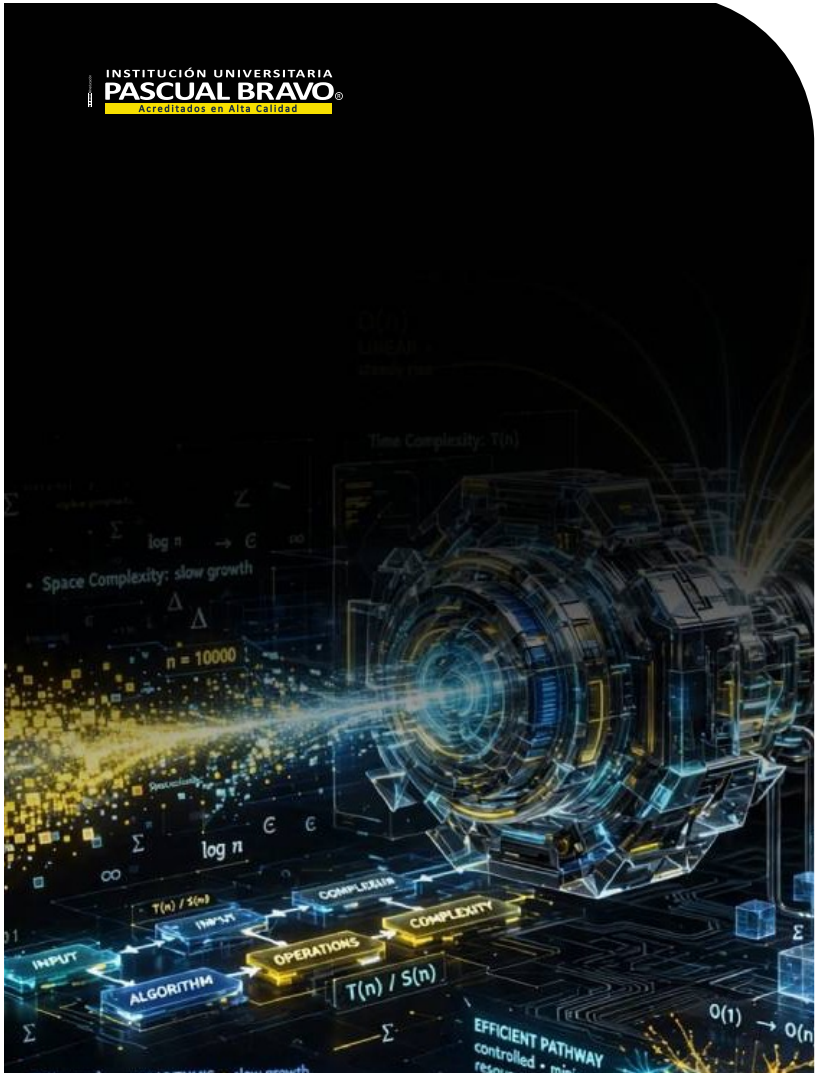


La complejidad computacional estudia los recursos necesarios para resolver un problema mediante un algoritmo, principalmente tiempo de **ejecución** y **memoria**, en función del tamaño de la entrada.

En programación, esto permite comparar soluciones que producen el mismo resultado pero tienen comportamientos muy diferentes cuando los datos crecen.

INSTITUCIÓN UNIVERSITARIA
PASCUAL BRAVO
Acreditados en Alta Calidad

Formar
Servir
Cuidar



https://youtu.be/o9nW0uBqvEo?t=492

https://ocw.mit.edu/courses/6-0001-introduction-to-computer-science-and-programming-in-python-fall-2016/resources/lecture-10-understanding-program-efficiency-part-1/

INSTITUCIÓN UNIVERSITARIA
PASCUAL BRAVO
Acreditados en Alta Calidad

Formar
Servir
Cuidar

Memoria y espacio

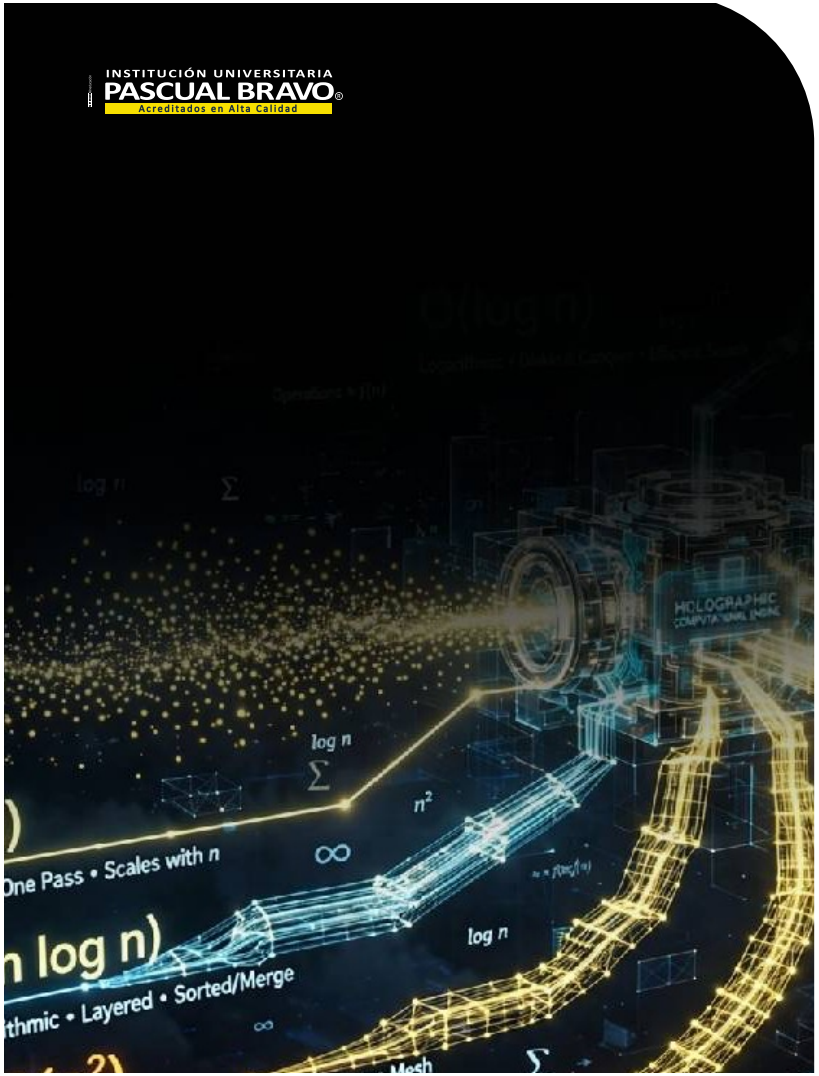


¿Qué pasa cuando definimos una variable?

$edad = 30;

Al declarer la variable, el programa necesita almacenar información y para ello usa un espacio de memoria donde se mantiene ese valor mientras el programa lo necesita.

https://learn.microsoft.com/en-us/dotnet/csharp/language-reference/language-specification/variables



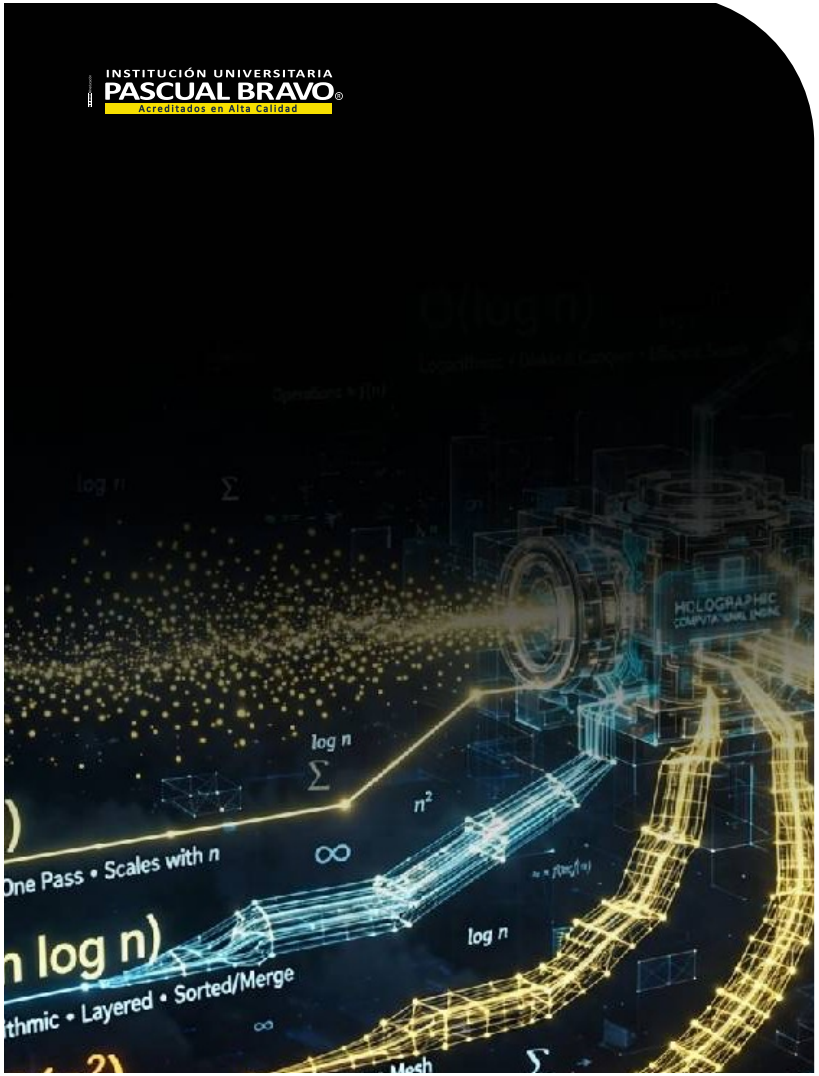
Formar
Servir
Cuidar

Variables de acción

Se puede pensar en las variables de acción como una memoria a corto plazo: sólo persisten durante la acción en curso.

Ejemplo: Se tiene un formulario e indica que ingrese su edad, La respuesta ingresada por el cliente se almacena automáticamente como una variable de acción denominada que será usada en un paso posterior de la misma acción, por ejemplo mostrar cierto contenido basado en la respuesta.

https://www.ibm.com/docs/es/watsonx/watson-orchestrate/base?topic=actions-using-variables-manage-conversation-information



Formar
Servir
Cuidar

Variables de session

Las variables de sesión son una memoria a largo plazo: permanecen activas durante toda la interacción del usuario con el asistente, y este puede acceder a ellas desde cualquier acción.

Ejemplo: Se inicia sesión en una app y cuando se cumplen las condiciones que el user y pass son correctos se almacenan datos en una variable de sesión con el fin de identificar quien es la persona que se encuentra activa en el sistema.

https://www.ibm.com/docs/es/watsonx/watson-orchestrate/base?topic=actions-using-variables-manage-conversation-information

INSTITUCIÓN UNIVERSITARIA
PASCUAL BRAVO
Acreditados en Alta Calidad

Formar
Servir
Cuidar

Notación Asintótica (Big O)



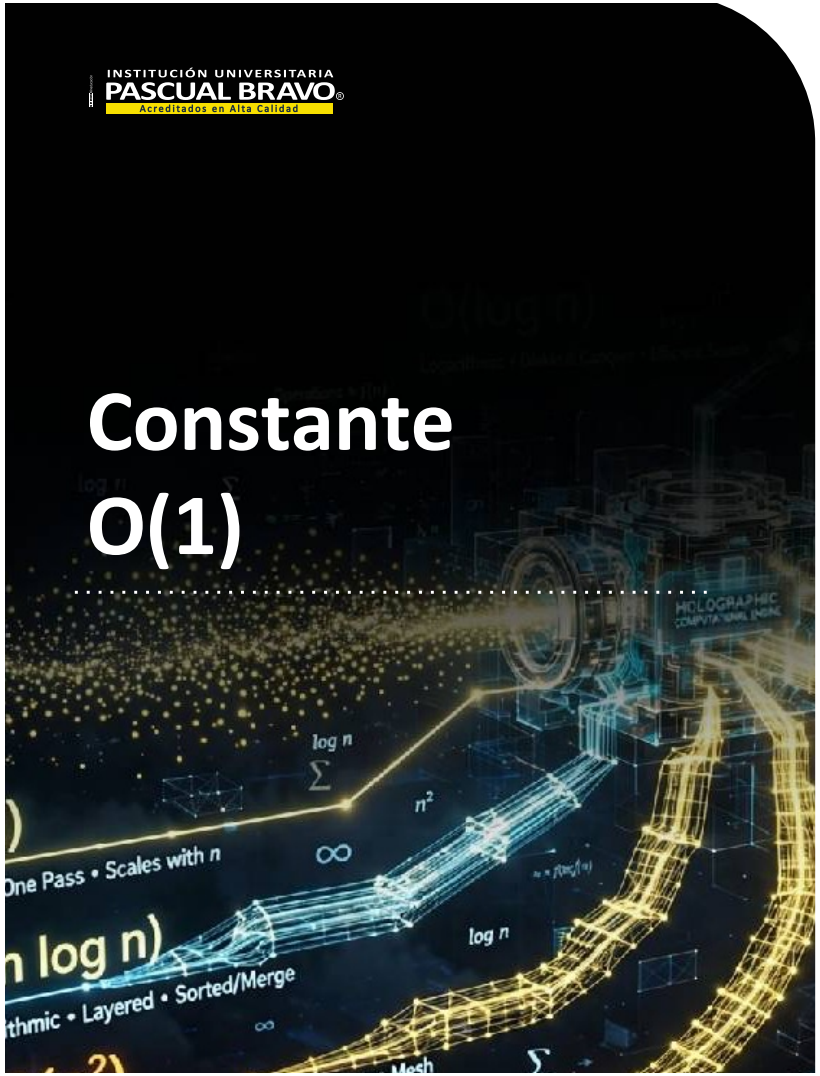
Es una herramienta matemática que permite describir cómo aumenta el costo de un **algoritmo** cuando aumenta la cantidad de datos que debe procesar.

https://ocw.mit.edu/courses/6-042j-mathematics-for-computer-science-spring-2015/mit6_042js15_textbook.pdf

INSTITUCIÓN UNIVERSITARIA
PASCUAL BRAVO
Acreditados en Alta Calidad

Formar
Servir
Cuidar

Constante
O(1)



No importa el **número de elementos** va a tardar el mismo número de tiempo

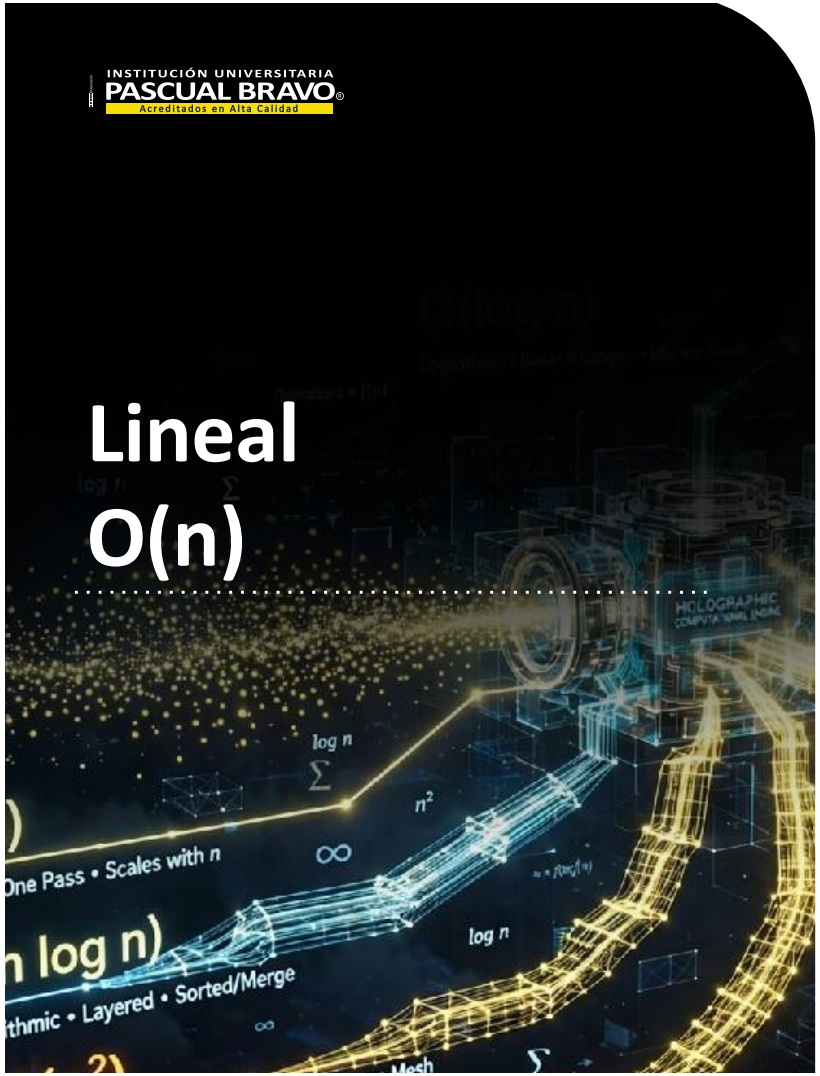
const personas = ["Ana", "Pedro", "Carlos", "Laura", "Juan"];

Console.log(**personas[1]**) // pedro

INSTITUCIÓN UNIVERSITARIA
PASCUAL BRAVO
Acreditados en Alta Calidad

Formar
Servir
Cuidar

Lineal
O(n)

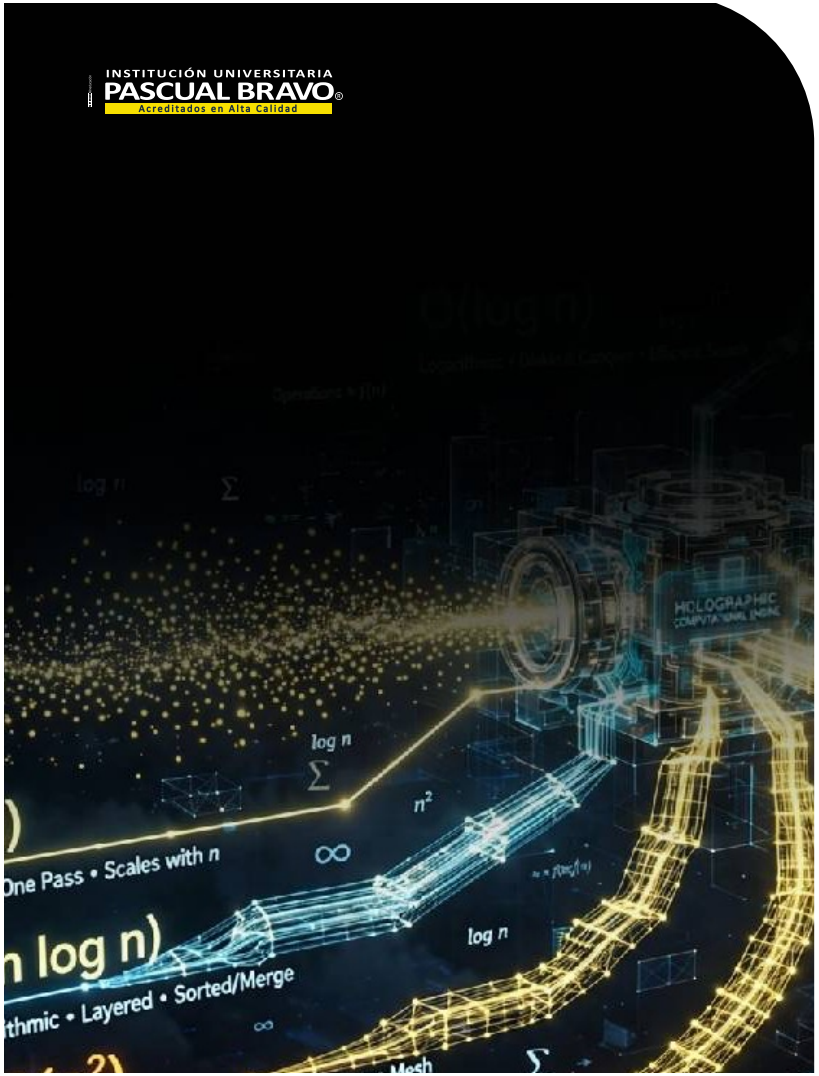


Significa que el costo de un algoritmo **crece aproximadamente** en proporción directa al **tamaño de la entrada**.

Se tiene el siguiente array

[🚗, 🧸, ⚽, 🟢, 🏀, 🚂, ...]

Pregunta:
¿Está el (pan 🍞) dentro del array?



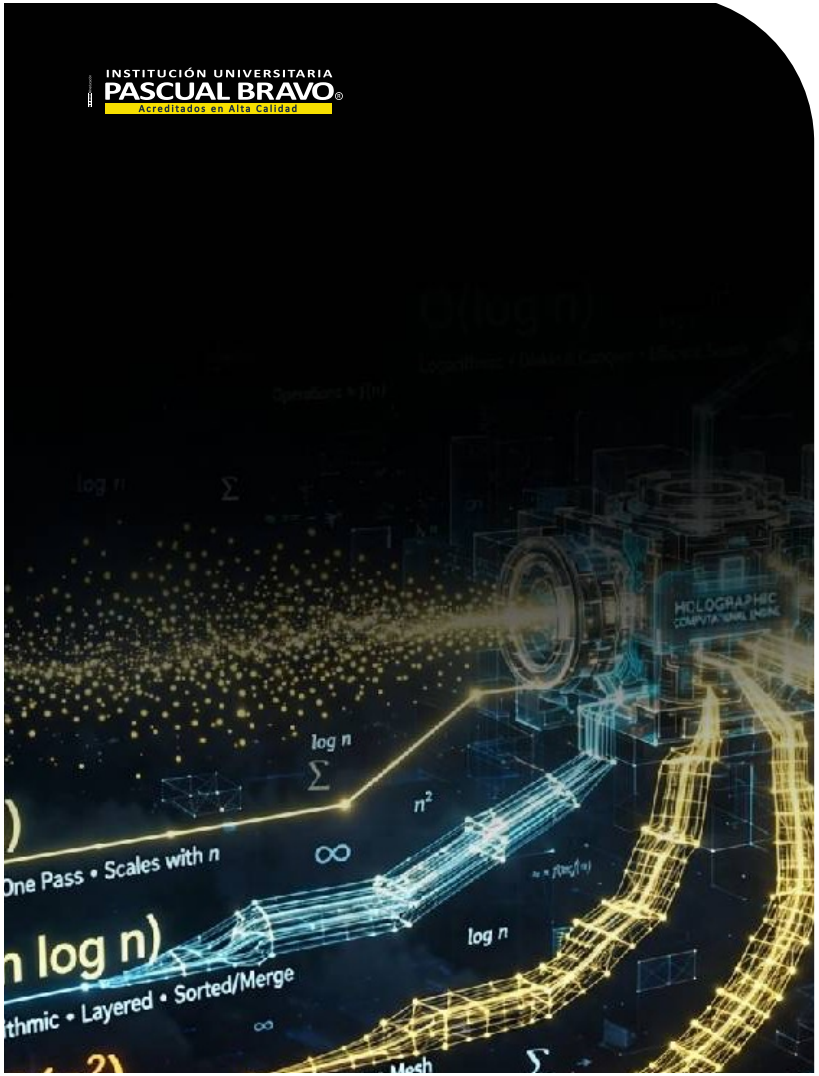

```
const personas = [ "Ana", "Pedro", "Carlos", "Laura", "Juan"];

function buscar(nombreABuscar, personas) {
  for (const persona of personas)
  {
        if (persona === nombreABuscar)
              return true;
  }
  return false;
}

buscar("Juan", personas);
```

El programa podría hacer:

Ana ❌
Pedro ❌
Carlos ❌
Laura ❌
Juan ✅

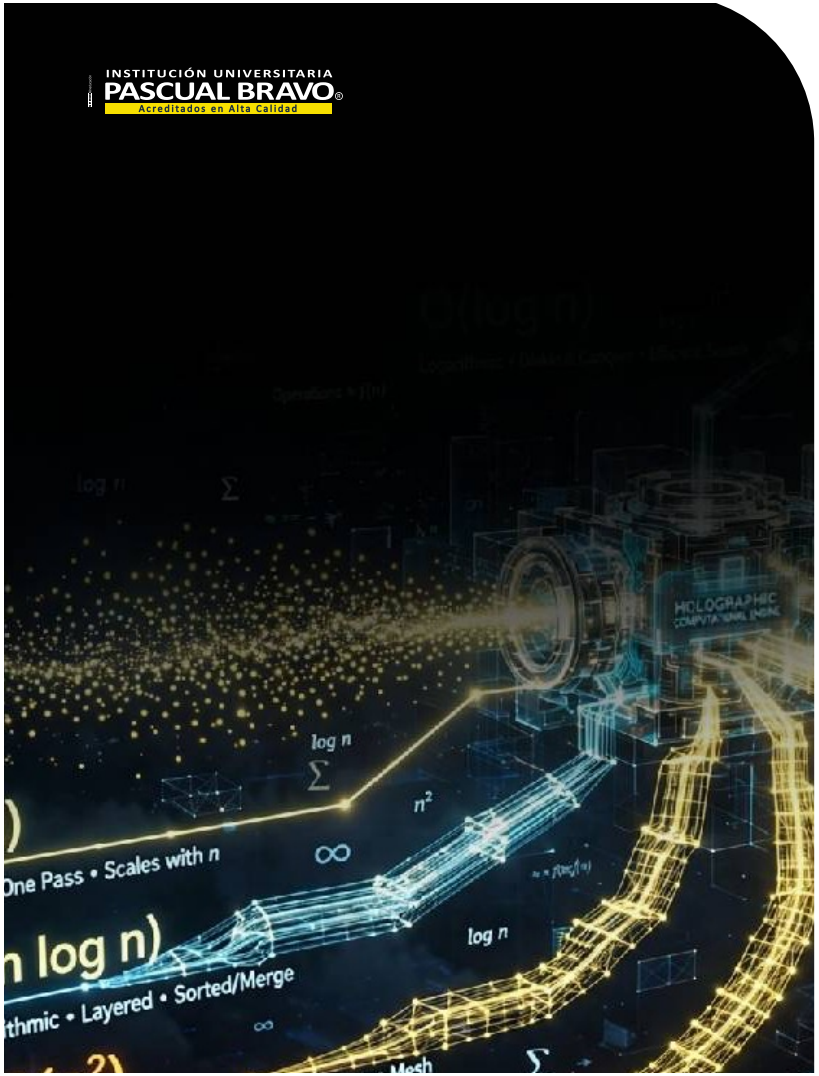


Formar
Servir
Cuidar

Pero si tenemos: 1.000, 5.000, 1.000.000 registros
Podríamos necesitar **n numeros de revisiones** hasta encontrar si existe o no.

Eso es lo que queremos expresar con: **O(n)**.

¿Qué significa la n?
El número de operaciones; crece aproximadamente en proporción al número de elementos.



¿Por qué es importante?
Porque un algoritmo puede funcionar perfectamente con 100 datos y convertirse en un problema con **100 millones**.

100 datos
↓
100 operaciones

1.000 datos
↓
1.000 operaciones

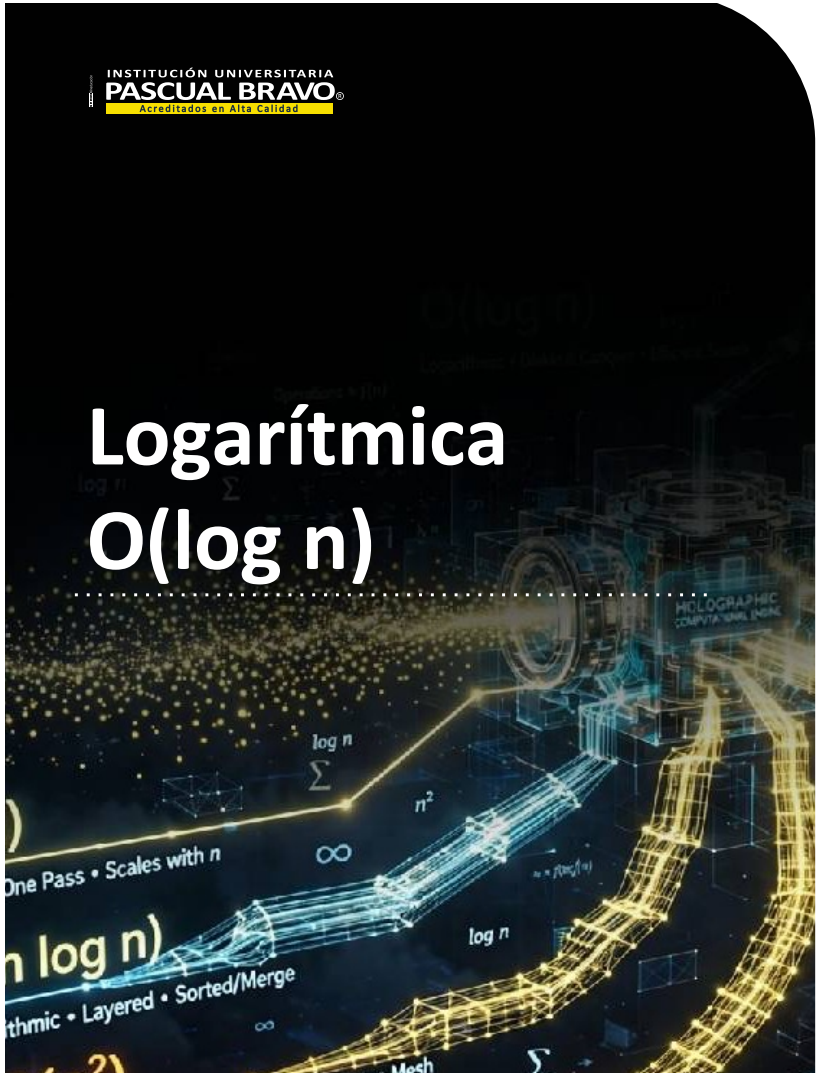
1.000.000 datos
↓
1.000.000 operaciones

100.000.000 datos
↓
100.000.000 operaciones

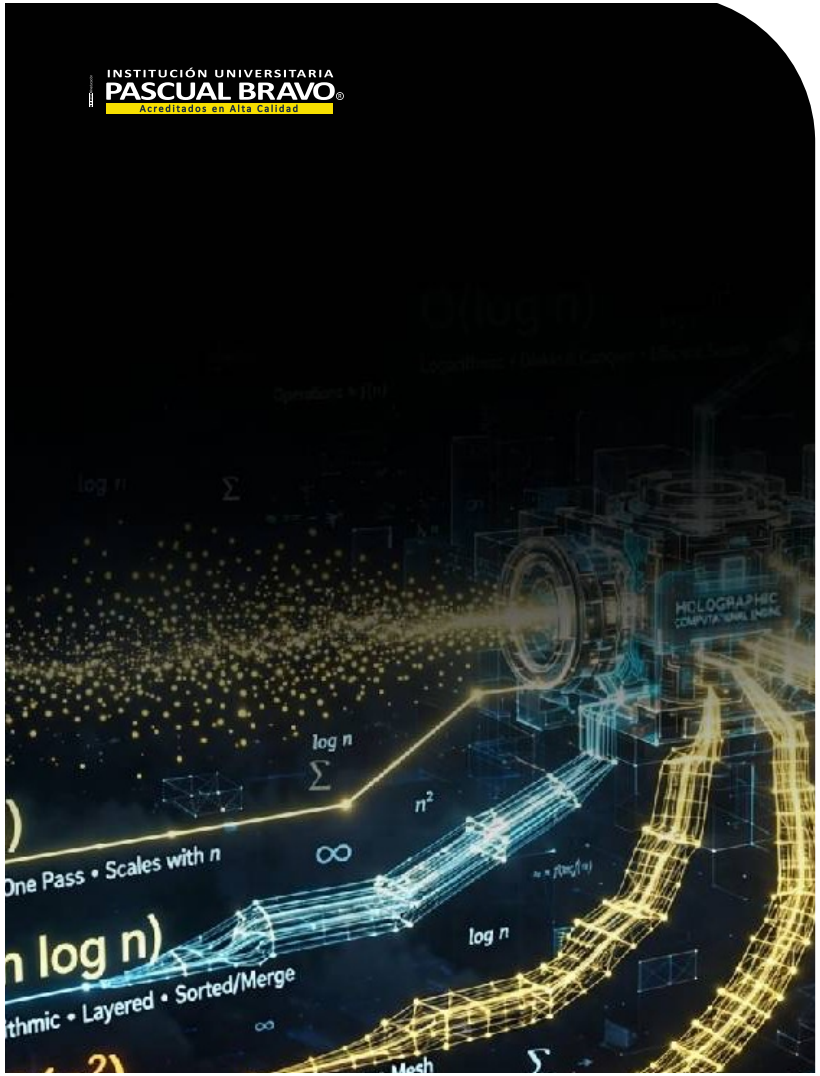
INSTITUCIÓN UNIVERSITARIA
PASCUAL BRAVO
Acreditados en Alta Calidad

Formar
Servir
Cuidar

Logarítmica
O(log n)

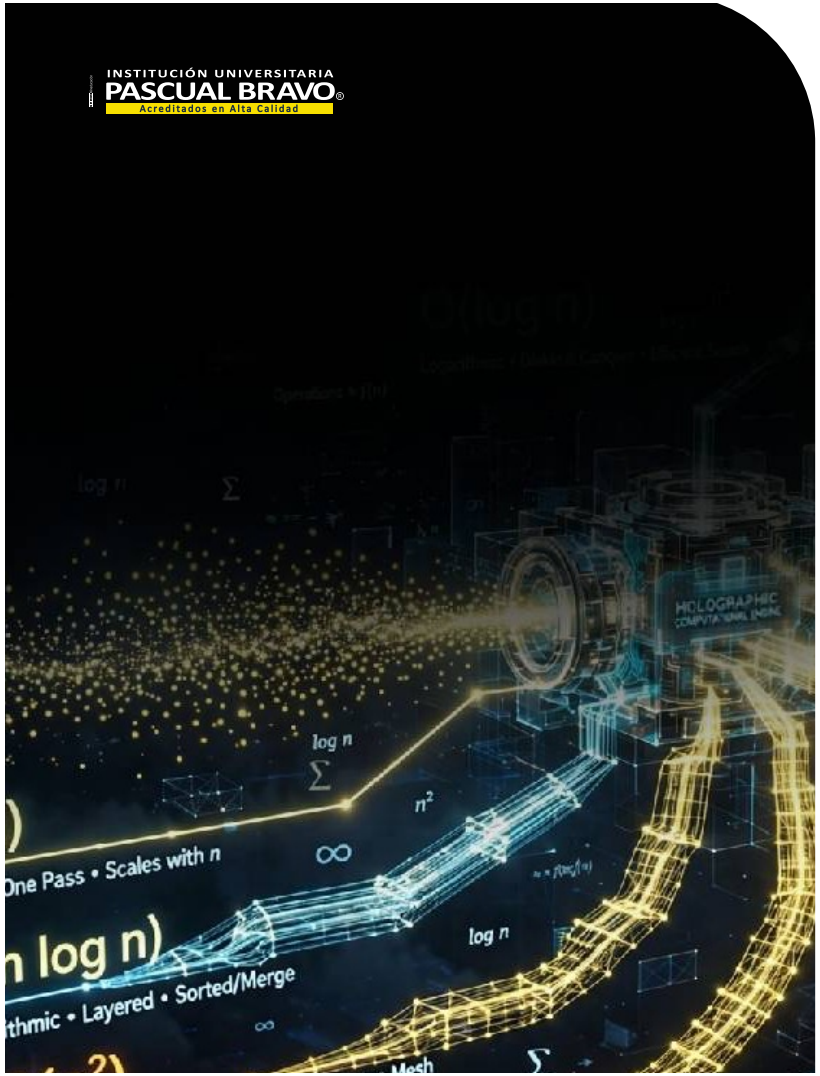


En lugar de revisar uno por uno, eliminamos una gran parte de los datos en cada paso.



Se tiene lo siguiente: [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]

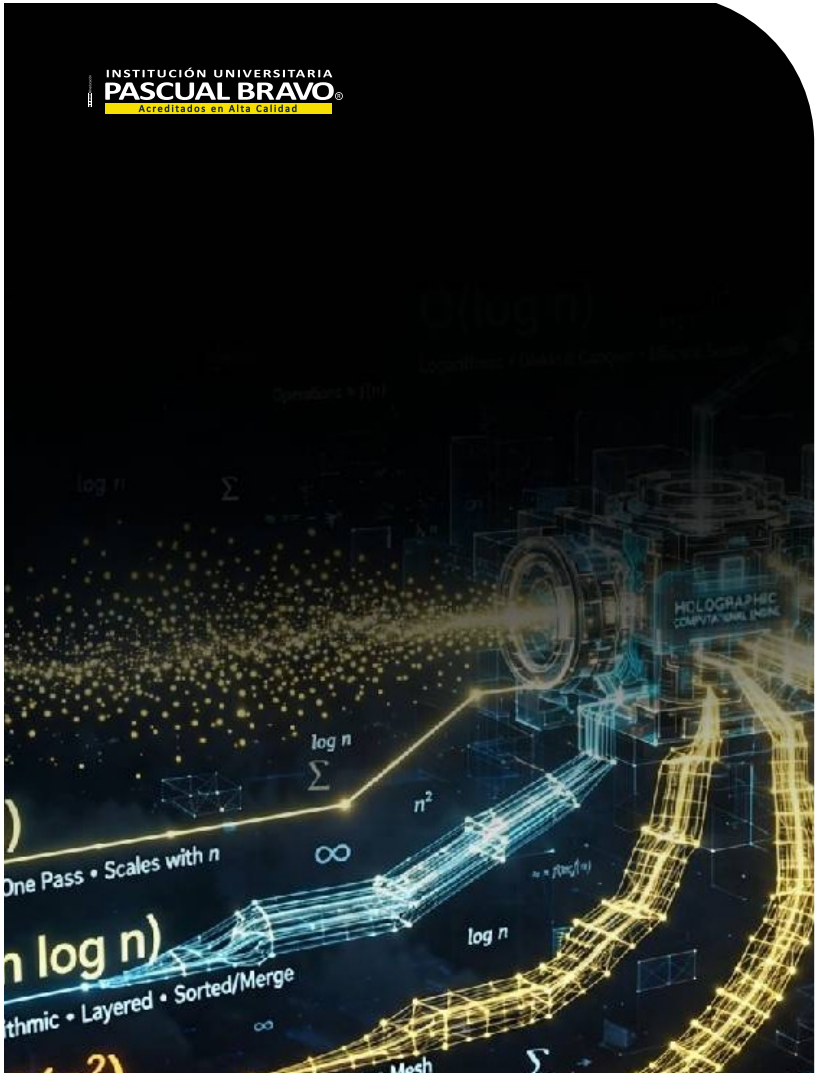
Buscamos el número **9**.



Formar
Servir
Cuidar

Primero miramos **aproximadamente** la mitad: 1 2 3 4 5 | 6 7 8 9 10

Como **9** es mayor que **5**, descartamos la **mitad izquierda**.



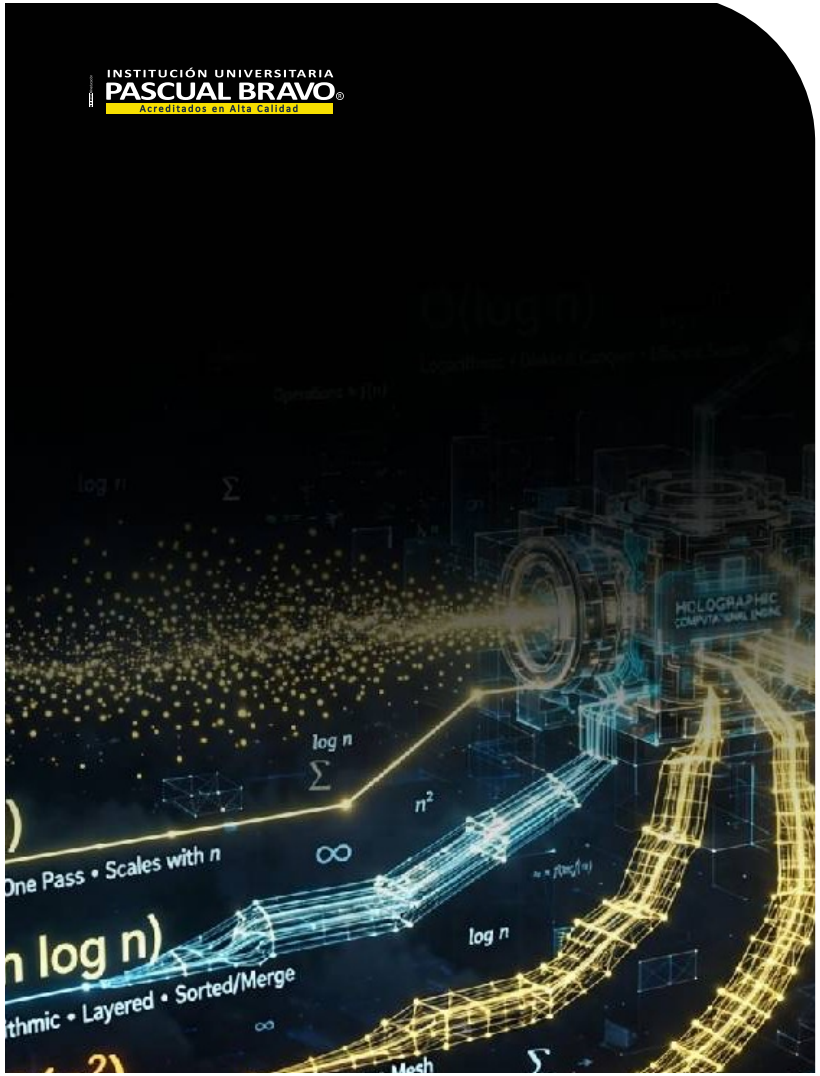
Ahora se tiene: 6, 7, 8, 9, 10

Volvemos a dividir: 6 7 | 8 9 10

Y seguimos.

Cada paso elimina **aproximadamente** la mitad de las posibilidades.

Por eso:
O(log n)

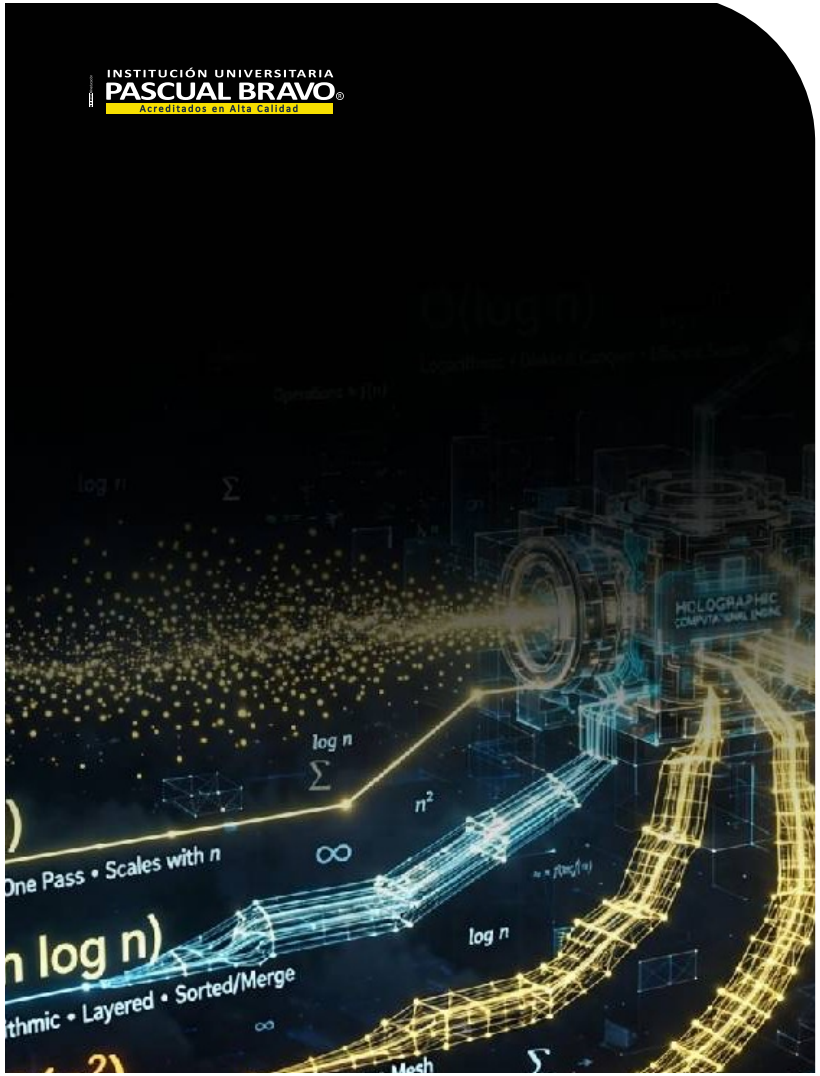


Formar
Servir
Cuidar

Si se busca valores cercanos al inicio o al fin, quizá la **búsqueda binaria** no sea siempre la mejor estrategia.

Por ejemplo, podrías usar una estrategia híbrida:

¿Está en las primeras o últimas posiciones?
↓
SÍ → búsqueda rápida
↓ NO
búsqueda binaria O(log n)



La búsqueda binaria requiere que los datos estén ordenados.

INSTITUCIÓN UNIVERSITARIA
PASCUAL BRAVO
Acreditados en Alta Calidad

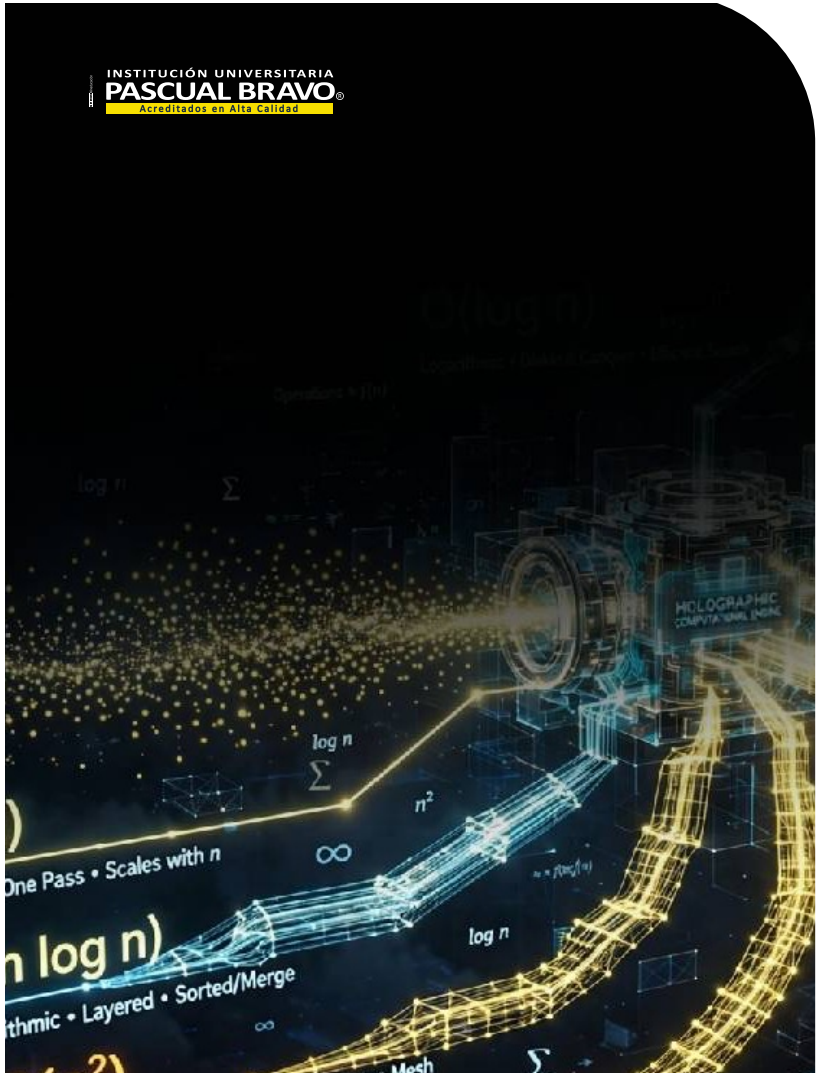
Formar
Servir
Cuidar

Cuadrática
$O(n^2)$



Cuando tenemos un ciclo dentro de otro ciclo (tabla de multiplicar).

```
for (const numero1 of numeros) {
   for (const numero2 of numeros) {
     console.log(numero1, numero2);
   }
}
```



Formar
Servir
Cuidar

https://colab.research.google.com/notebook#create=true

```
import time

valor = 10

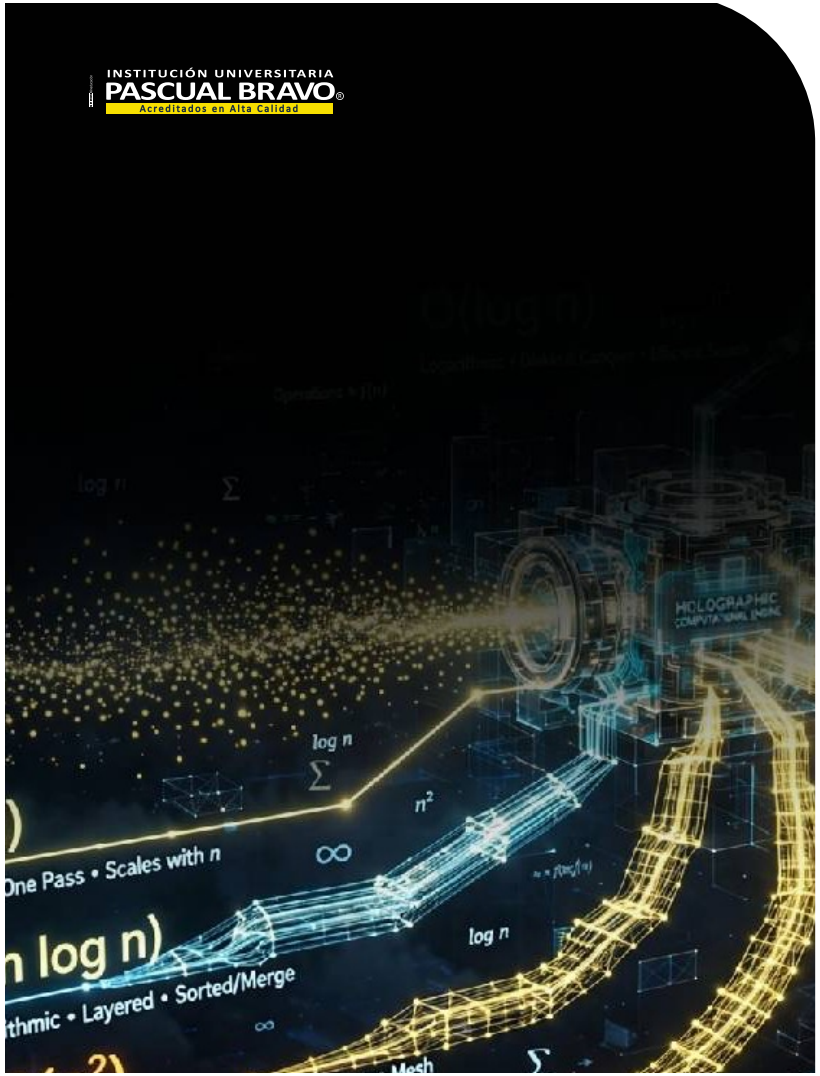
inicio = time.time()

for i in range(1, valor + 1):
    for j in range(1, i + 1):
        print(f"{i} x {j} = {i * j}")

fin = time.time()

tiempo_segundos = fin - inicio

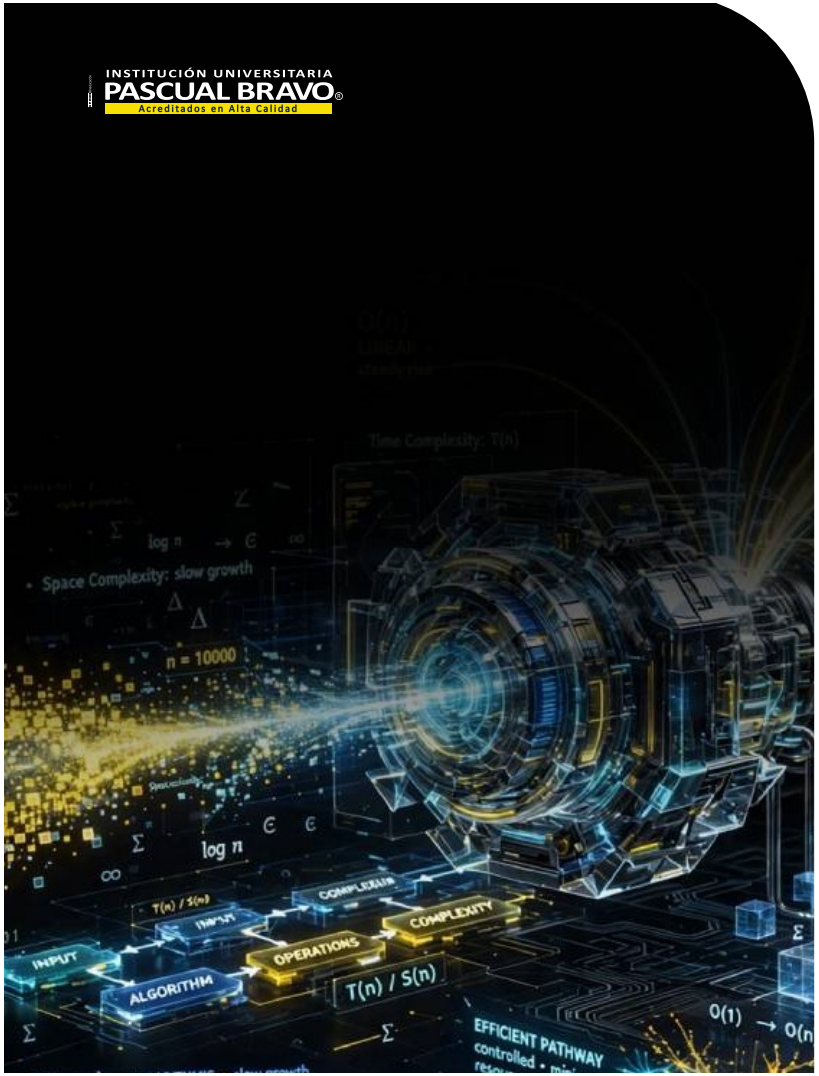
print(f"Tiempo: {tiempo_segundos} segundos")
```



Formar
Servir
Cuidar

Supongamos
valor = 5

i = 1 → j = 1	→ 1 operación
i = 2 → j = 1, 2	→ 2 operaciones
i = 3 → j = 1, 2, 3	→ 3 operaciones
i = 4 → j = 1, 2, 3, 4	→ 4 operaciones
i = 5 → j = 1, 2, 3, 4, 5	→ 5 operaciones



Cuando te enfrentes a un algoritmo, intenta hacer siempre estas preguntas:

- ¿Cuál es el tamaño de la entrada? → n
- ¿Cuántas veces se ejecuta cada operación?
- ¿Hay bucles anidados?
- ¿La entrada se divide?
- ¿Hay recursividad?
- ¿Se crean estructuras adicionales?
- ¿Cómo crece el costo cuando n aumenta?

https://ocw.mit.edu/courses/6-0001-introduction-to-computer-science-and-programming-in-python-fall-2016/resources/lecture-10-understanding-program-efficiency-part-1/

Si conocemos el **primer número de una secuencia** y **cuánto aumenta o disminuye en cada paso**

¿Cómo podemos descubrir qué valor tendrá cualquier elemento de la secuencia sin tener que calcular todos los anteriores?

INSTITUCIÓN UNIVERSITARIA
PASCUAL BRAVO
Acreditados en Alta Calidad

Formar
Servir
Cuidar

Progresiones aritméticas



$$a_n = a_1 + (n-1)d$$

La fórmula del término **n** permite encontrar directamente cualquier elemento de una sucesión aritmética, sin tener que calcular todos los términos anteriores.

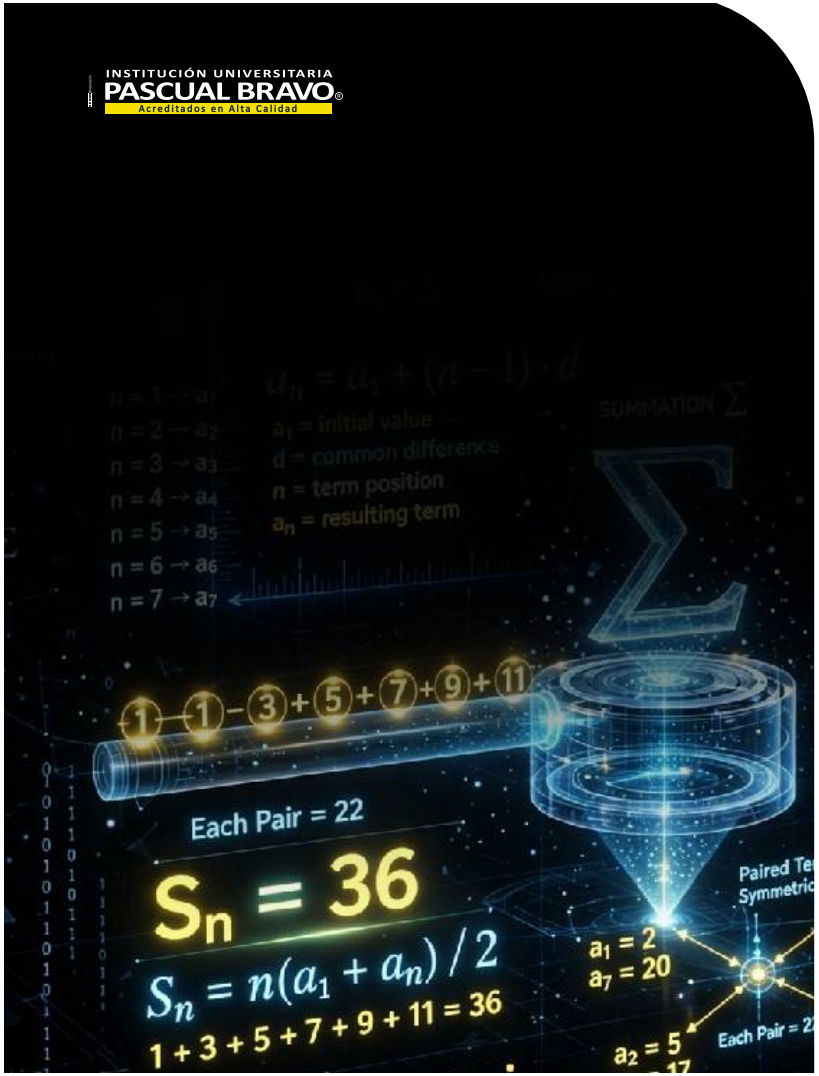
2 2 2 2 2
2, 4, 6, 8, 10, 12 — Cada número aumenta en: 2

-10 -10 -10 -10 -10
100, 90, 80, 70, 60 — Cada número disminuye en: 10

La cantidad que agregamos o restamos se llama **diferencia común** y la representamos con: **d**

Sólo funciona cuando existe una diferencia común.

https://openstax.org/books/algebra-1/pages/4-18-3-define-an-arithmetic-sequence-by-the-nth-term



Formar
Servir
Cuidar

POS 10

3, 6, 9, 12, 15, 18, 21, 24, 27, **?**

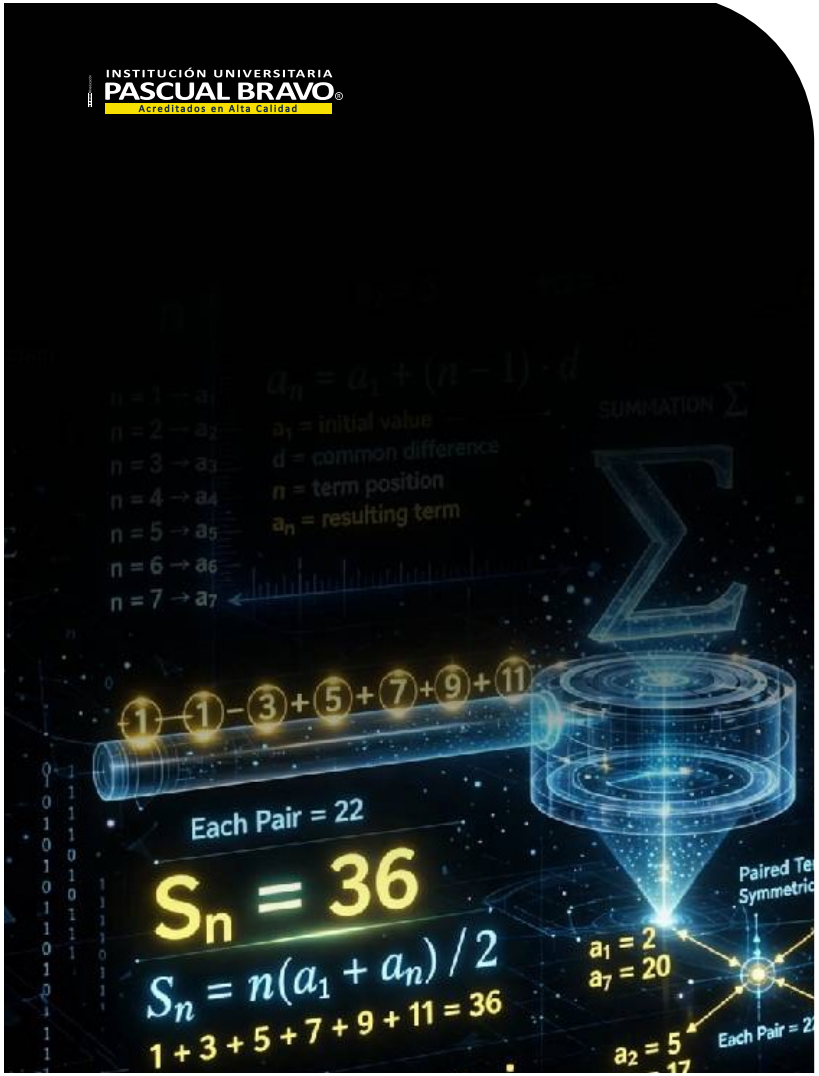
$a_n=a_1+(n-1)d$

a_1 = primer elemento
n = **posición que queremos encontrar**
d = diferencia común
a_n = elemento que buscamos

$a_1 = 3$ y $d = 3$

¿Cuál es el elemento número 10?

$a_{10} = 3+(10-1)(3)$
$a_{10} = 3+((9)(3))$
$a_{10} = 3+27$
$a_{10} = 30$



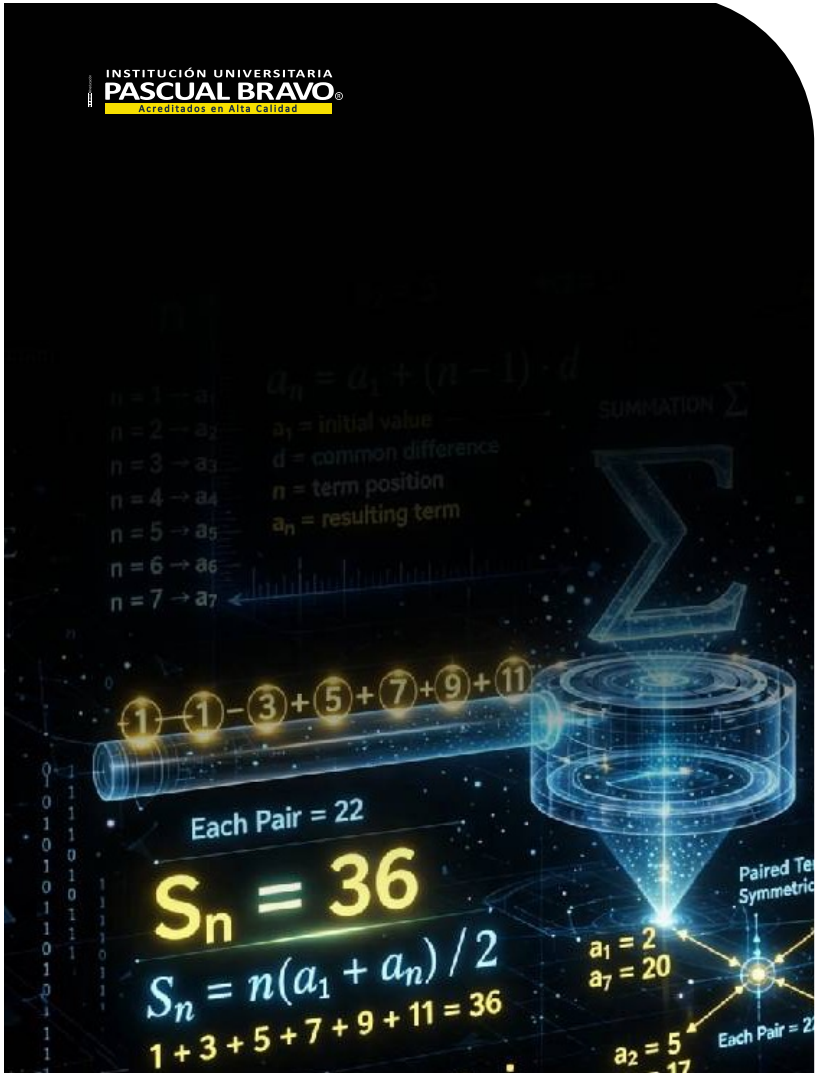
Formar
Servir
Cuidar

¿Qué tiene que ver esto con algoritmos?

Supongamos este código:

```
for i in range(1, 11):
    print(i)
```

Realiza 10 operaciones
1, 2, 3, 4, 5, 6, 7, 8, 9, 10



Formar
Servir
Cuidar

```
for i in range(1, 11):
    for j in range(1, i + 1):
        print(i, j)
```

La cantidad de operaciones es:
i = 1 → 1
i = 2 → 2
i = 3 → 3
i = 4 → 4
...
i = 10 → 10

INSTITUCIÓN UNIVERSITARIA
PASCUAL BRAVO
Acreditados en Alta Calidad

Formar
Servir
Cuidar

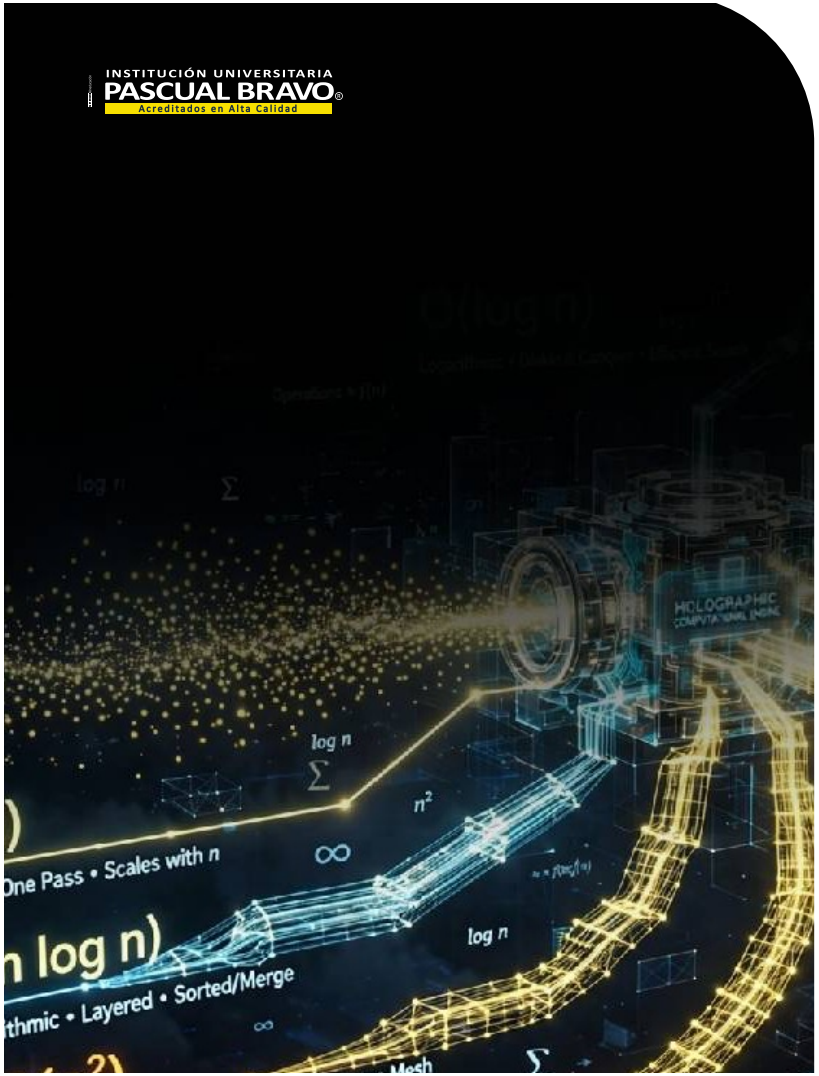
Recursividad



Una función recursiva es una función que se llama a sí misma.

```
function funcionRecursiva(parametro) {
   // CASO BASE
   if (condicionBase) return valorBase;

   // CASO RECURSIVO
   return funcionRecursiva(parametroModificado);
}
```




```
function factorial(n:number):number {
    // CASO BASE
    if (n <= 1)
            return 1;

    // CASO RECURSIVO
    return n * factorial(n - 1);
}


console.log(factorial(1)); // 1
console.log(factorial(5)); // 120
console.log(factorial(10)); // 3628800
```

PASCUAL BRAVO



Formar • Servir • Cuidar • Formar • Servir • Cuidar •

Ejercicio

Appresso (Tu café, a un tap)

Contexto:
Una cafetería digital procesa diariamente cientos o miles de pedidos realizados desde una aplicación. Cada pedido contiene información como identificador, cliente, productos, cantidades y estado (**Solicitado, En Proceso, Entregado**). El sistema necesita buscar pedidos, calcular estadísticas y procesar información de manera eficiente.

Inicialmente, se desarrolló una solución que funciona correctamente **con pocos pedidos**; a medida que ha pasado el tiempo, se empezaron a presentar problemas de rendimiento en el sistema y mucho más a medida que aumenta el volumen de información.

Appresso (Tu café, a un tap)

Objetivo:
Analizar el comportamiento de diferentes algoritmos utilizando complejidad computacional, Big O, progresiones aritméticas, sumatorias y recursividad.
Se deberá determinar cómo **aumenta el número de operaciones** cuando crece la cantidad de pedidos, identificar algoritmos lineales o cuadrático, representar operaciones mediante sumatorias, analizar una función recursiva y comparar los resultados con mediciones realizadas mediante código.
El objetivo final es justificar técnicamente qué solución resulta más eficiente y escalable

Consideraciones:
Realiza pruebas con diferentes tamaños de entrada, por ejemplo 10, 100, 1.000, 10.000 y 100.000 pedidos. No basar el análisis únicamente en el tiempo de ejecución:

- Contabiliza también las operaciones realizadas
- Identifica el mejor y peor caso cuando corresponda.

INSTITUCIÓN UNIVERSITARIA PASCUAL BRAVO
Acreditados en Alta Calidad

Formar Servir Cuidar

Appresso (Tu café, a un tap)

Casos:

1. El sistema **recibe** una lista de pedidos y debe realizar operaciones principales:
   - Encontrar un pedido específico,
   - Calcular el total de productos procesados
   - Analizar una estructura de pedidos utilizando una función recursiva.

2. Ejecutar los algoritmos utilizando diferentes cantidades de pedidos, contar sus operaciones, identificar patrones mediante progresiones, sumatorias y determinar la **complejidad temporal** utilizando notación Big O.

3. Una cliente durante algún tiempo ha venido incrementando sus pedidos de manera algo particular, inicio con **dos** productos a la siguiente semana **cuatro** productos y así durante el último mes; determina en que momento esta empresa llegará a solicitar **42, 72 y 120** productos, ya que al llegar a estos límites se le podrá brindar un descuento (x), de esta forma la empresa podrá determinar si genera un **sistema de fidelización** por cliente.
4. **"Predecir"** usando Regresión lineal de como serian las posibles ventas en 2,5 ó 7 días partiendo del dia actual.

Appresso (Tu café, a un tap)

Complejidad computacional

1. Implementa una función que busque un pedido mediante su identificador y analiza qué ocurre cuando la cafetería procesa: 10, 100, 1.000, 10.000, n pedidos.
2. Explica cómo aumenta el costo computacional cuando aumenta n y determina la complejidad temporal mediante Big O.